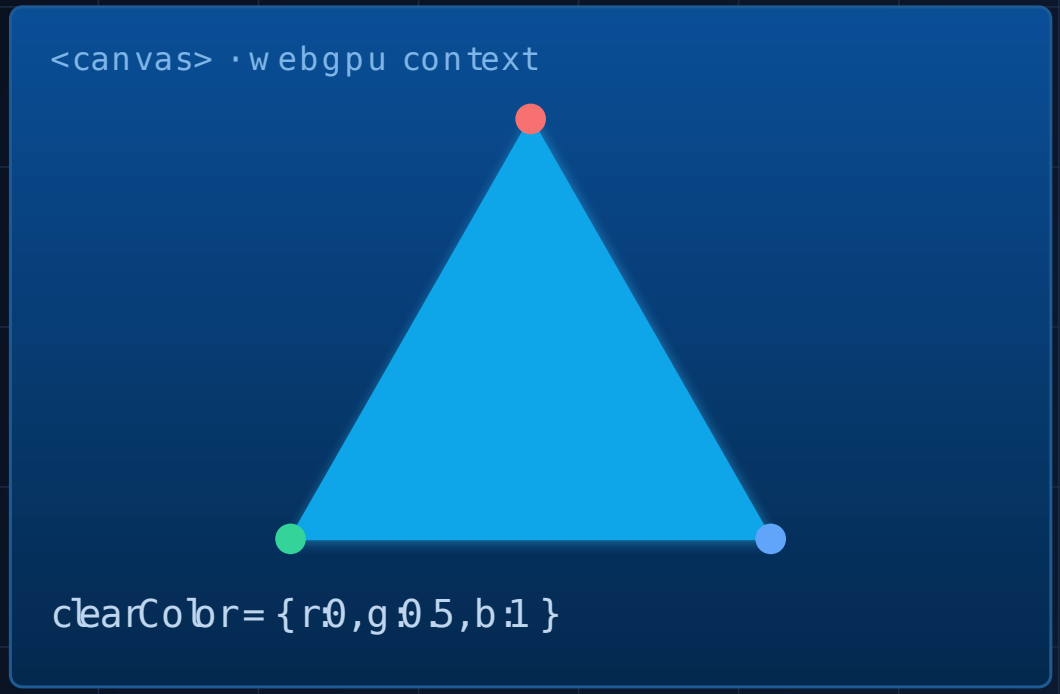
MDN / WEB API / WEBGPU

WebGPU 是什么

WebGL 的继任者
把现代 GPU 能力带进网页

面向前端工程师的入门简报

事实来源 :MDN Web Docs — WebGPU API 概览页



三个顶点各带一种颜色，片元着色器把它们插值成整块渐变。

为什么会有 WebGPU

WebGL 的三个根本问题

01 拿不到现代 GPU 新特性

Direct3D 12 、 Metal 、 Vulkan 提供大量新功能 ;
OpenGL 与 WebGL 已无更新计划 , 不会再拿到。

02 对通用计算支持很差

WebGL 完全围绕 “画到 canvas” 这一用例设计 ;
而机器学习等 GPGPU 场景越来越重要。

03 3D 应用越来越吃资源

同屏需要渲染的对象越来越多 ,
新的渲染特性也在不断加码。

WebGPU 的回应

- 与现代 GPU API 对齐的通用架构
- 一等公民的 GPGPU 计算支持
- 单个对象渲染的**CPU 开销显著更低**
- 计算粒子、后处理滤镜等现代特性
- 剔除、蒙皮变换可直接在 GPU 上做

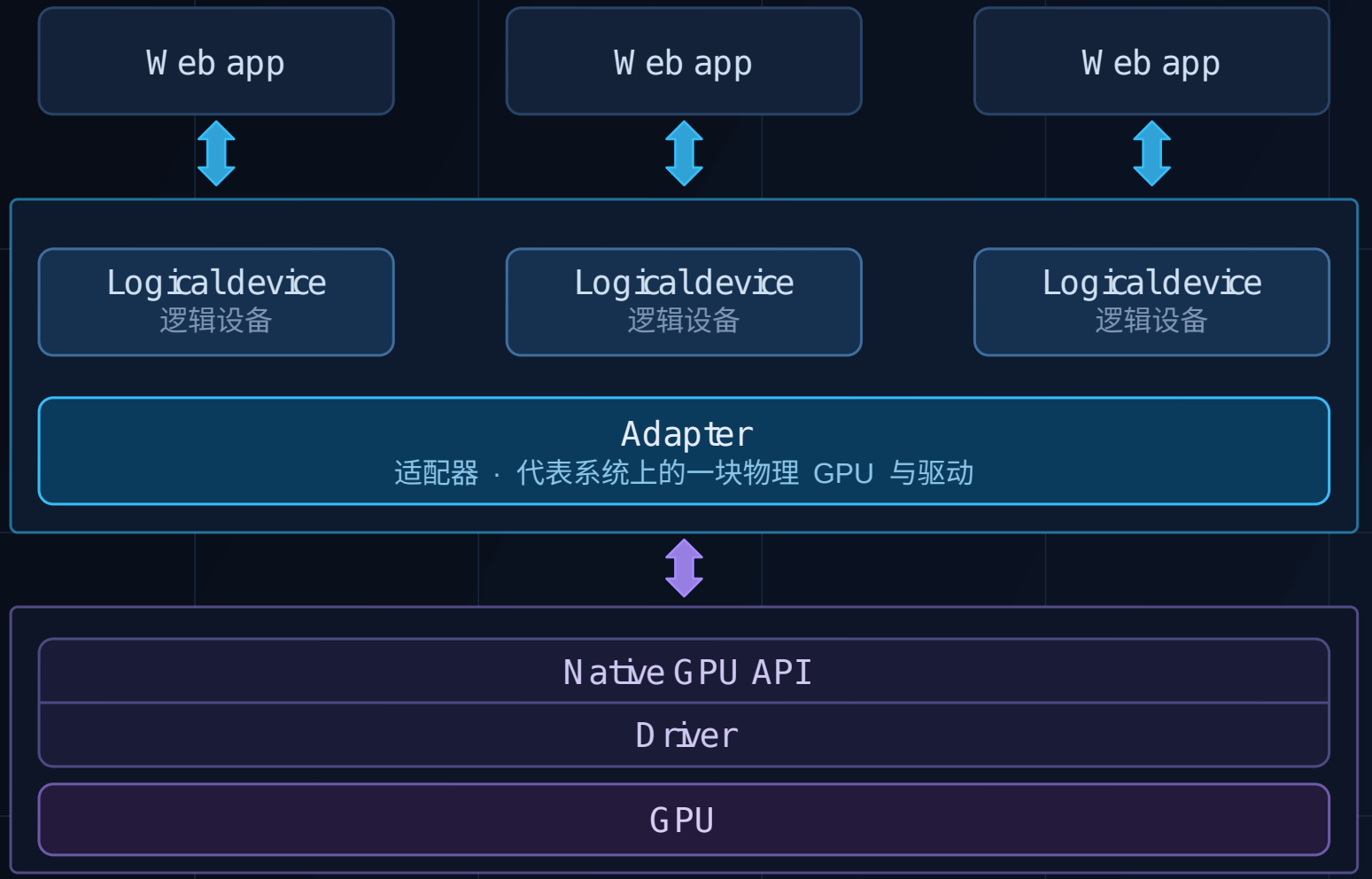
来源 :MDN 《 WebGPU API 》 Concepts and usage

02 / 分层模型

从网页到 GPU，中间隔着几层

WebGPU
浏览器实现

底层系统
操作系统与硬件



同一块 GPU 会被多个应用和进程并发使用。

逻辑设备提供多路复用，让每个应用彼此隔离。

图中按“只有一块 GPU、一套原生 API 和驱动”来画。

三步拿到设备 :gpu → adapter → device

navigator.gpu
取得当前上下文的 GPU 对象

↓

GPU.requestAdapter()
取得适配器，不传参就是默认适配器

↓

GPUAdapter.requestDevice()
取得逻辑设备，可用描述符指定 features 与 limits

两个方法的参数都可以省略，默认值对多数场景已经够用。

- GPUDevice 是访问一切功能的起点
- 在 Worker 里改用 WorkerNavigator.gpu

来源 :MDN 《WebGPU API》 Accessing a device

js · 特性检测 + 取设备

```
async function init(){
if(!navigator.gpu){
throw Error("WebGPU not supported.");
}
let adapter;
try{
adapter=await navigator.gpu.requestAdapter();
}catch (error){
console.error(error);
}
if(!adapter){
throw Error("Couldn't request WebGPU adapter.");
}
const device=await adapter.requestDevice();
}
```

core 与 compatibility: 两档能力

core 默认档

默认情况下，一个 GPUAdapter 支持全部 core WebGPU 特性与限制。

面向拥有现代平台图形 API 的设备。

检测方式：适配器或设备是否带 core-features-and-limits

compatibility 可选档

只支持能在 OpenGL ES 3.1、Direct3D 11 等更老图形 API 上运行的**受限子集**。

受限应用仍然是合法的 core 应用，因此在所有支持 core WebGPU 的浏览器上都能运行。

js · 请求兼容模式适配器

```
const adapter = await navigator.gpu
.requestAdapter({ featureLevel: "compatibility" });
```

js · 判断当前跑在哪一档

```
const isCore = device.features
.has("core-features-and-limits");
```

来源：MDN 《WebGPU API》 WebGPU compatibility mode

WebGPU 目前有两类管线

管线是一个包含若干可编程阶段的逻辑结构，这些阶段跑完，程序的活就干完了。

渲染管线

renderpipeline

把图形画出来，通常画进 <canvas>，也可以离屏渲染。

vertex stage

顶点着色器按位置数据在 3D 空间摆放顶点，装配成三角形等图元，再由 GPU 光栅化，算出每个图元覆盖画布上的哪些像素。

fragment stage

片元着色器为图元覆盖的每个像素算颜色，常以纹理提供表面细节与虚拟光源的位置颜色。

计算管线

compute pipeline

用于通用计算，只包含一个阶段。

compute stage

计算着色器拿到通用数据，按指定的工作组数量并行处理，再把结果写回一个或多个 buffer。

这些 buffer 可以装 **任何类型** 的数据。

计算管线不画东西，所以不需要指定图元类型。

着色器是 GPU 处理的一组指令，用类 Rust 的低级语言 **WGSL**(WebGPU Shading Language) 编写。

顶点阶段与片元阶段，以及写它们的 WGSL

渲染管线

renderpipeline

vertex stage

顶点着色器拿到喂给 GPU 的位置数据，在 3D 空间里摆放一串顶点，可以施加旋转、平移、透视等效果；顶点随后被装配成三角形等图元——渲染图形的基本积木——再由 GPU 光栅化，算出每个图元覆盖画布上的哪些像素。

fragment stage

片元着色器为上一阶段产出的图元所覆盖的每个像素计算颜色；这些计算常用到纹理形式的图像输入，提供表面细节与虚拟光源的位置颜色。

两个阶段的入口函数名在管线描述符里指定。

wgsl · 一个着色器模块里的两个阶段

```
struct VertexOut {
@builtin(position) position : vec4f,
@location(0) color : vec4f
}
@vertex
fn vertex_main(@location(0) position: vec4f,
@location(1) color: vec4f) -> VertexOut
{
var output : VertexOut;
output.position = position;
output.color = color;
return output;
}
@fragment
fn fragment_main(fragData: VertexOut) -> @location(0) vec4f
{
return fragData.color;
}
```

着色器代码常放在模板字符串或 <script> 里读取；WGSL 的 mime 类型是 text/wgsl。

一个 WebGPU 应用大致要走五步

01 **着色器模块**

用 WGSL 写好着色器代码，打包进一个或多个 shader module。

02 **画布上下文**

拿到 <canvas> 的 webgpu 上下文并配置它；纯计算的程序不需要这步。

03 **数据资源**

要被管线处理的数据，得先放进 GPU buffer 或 texture 里才能被访问。

04 **创建管线**

用描述符写清数据结构、绑定、着色器与资源布局，再据此创建管线。

05 **跑一个 pass**

创建编码器、下达命令、封装成命令缓冲，最后提交到设备的命令队列。

第 05 步展开：一个 pass 的五个子步骤

`commandEncoder` 创建命令编码器 → `passEncoder` 创建 pass 编码器 → `commands` 下达绘制命令 → `commandBuffer` 封装命令缓冲 → `queue.submit()` 提交给 GPU

基础 demo 只有一条管线；复杂应用通常会为不同用途准备多条管线。

一趟渲染 pass，逐行读下来

js · 基础渲染 demo 的渲染流程

```
const commandEncoder = device.createCommandEncoder();
const clearColor = { r: 0.0, g: 0.5, b: 1.0, a: 1.0 };
const renderPassDescriptor = {
colorAttachments: [
{
clearValue: clearColor,
loadOp: "clear",
storeOp: "store",
view: context.getCurrentTexture().createView(),
},
],
};
const passEncoder =
commandEncoder.beginRenderPass(renderPassDescriptor);
passEncoder.setPipeline(renderPipeline);
passEncoder.setVertexBuffer(0, vertexBuffer);
passEncoder.draw(3);
passEncoder.end();
device.queue.submit([commandEncoder.finish()]);
```

① 视图会先被清成 clearColor，三角形背后那块纯蓝就是这么来的；storeOp 为 "store" 表示这一趟的结果要存下来。

② setPipeline 指定这一趟用哪条管线；setVertexBuffer 的第一个参数是槽位，对应顶点缓冲布局数组里的下标。

③ vertexBuffer 里放着三个顶点的数据，所以这里是 **draw(3)** 。

④ end() 结束命令列表，finish() 封装成 command buffer，再由 submit() 交给设备的命令队列。

09 / 计算管线

算完的结果，怎么回到 JavaScript

js · 基础计算 demo 的两个 buffer 与读回

```
const output = device.createBuffer({
size: BUFFER_SIZE,
usage: GPUBufferUsage.STORAGE | GPUBufferUsage.COPY_SRC,
});
const stagingBuffer = device.createBuffer({
size: BUFFER_SIZE,
usage: GPUBufferUsage.MAP_READ | GPUBufferUsage.COPY_DST,
});
commandEncoder.copyBufferToBuffer(
output, 0, stagingBuffer, 0, BUFFER_SIZE);
device.queue.submit([commandEncoder.finish()]);
await stagingBuffer.mapAsync(GPUMapMode.READ, 0, BUFFER_SIZE);
const copyArrayBuffer = stagingBuffer.getMappedRange(0, BUFFER_SIZE);
const data = copyArrayBuffer.slice();
stagingBuffer.unmap();
console.log(new Float32Array(data));
```

GPU buffer 紧贴 GPU 核心存放，浏览器这样的主机进程读不到，所以要中转。



读完记得 unmap()
把 staging buffer 交还回去。

错误会传染，所以要圈作用域来抓

错误是“传染性”的

callA
被判定非法
→
callB
用了 A 的返回值
→
callC
用了 B 的返回值

WebGPU 的调用在 GPU 进程里异步校验；某次调用一旦被标记为无效，依赖它返回值的对象也会跟着无效，一路传下去 —— 所以这类错误被称为 contagious。

每个设备有自己的作用域栈

```
device.pushErrorScope("validation");
// … 你要检查的那批调用 …
const err = await device.popErrorScope();
```

`GPUValidationError` · `GPUOutOfMemoryError` · `GPUInternalError`

弹出作用域后拿到的 Promise，解析成该作用域内捕获到的第一个错误；没有错误则为 `null`。

对比：WebGL 历史上的做法是同步的 `getError()`

同步返回，伤性能

每次调用都要往返一次 GPU，并且要求此前发出的所有操作都已完成。

状态模型是扁平的

错误会在互不相关的代码之间泄漏。WebGPU 的设计者决心改掉这一点。

今天能用到什么程度

Limited availability

该特性不是 Baseline，因为它在一些使用最广泛的浏览器里无法工作。

Secure context

只在安全上下文 (HTTPS) 里可用 —— 在部分或全部支持它的浏览器中如此。

具体版本号 :NO DATA

MDN 的兼容性表由脚本渲染，本次抓取到的页面内容里没有版本数据。

另一处 **NO DATA** :compatibility mode 的确切限制清单 —— 本页只给出外部文档链接，没有列出内容。

先做能力检测，再写渲染代码

拿不到 `navigator.gpu` 或适配器时直接走降级路径 —— 这是现状唯一能落到代码上的结论。

本篇全部事实来自单一来源 :MDN Web Docs 《WebGPU API》概览页 (页面最后修改于 2026 年 9 月 2 日) 。

developer.mozilla.org/en-US/docs/Web/API/WebGPU_API